

Habilidades e formação

Quais habilidades fazem alguém ser bom em programar usando IA?

O uso da IA ajuda ou atrapalha quem está aprendendo?

Integrantes

Enrico Bergonzo · Felipe Formolo · Franco Scotti · Gabriel Custodio · Larissa Rodrigues

Uma discussão sobre fundamentos, aprendizagem, dependência e o papel humano na engenharia de software.

Cinco ângulos para responder à mesma questão

Da proficiência com IA à formação do profissional que precisa supervisioná-la.

- | | | |
|----|------------------------------|---|
| 01 | Vibe coding | O que realmente prediz desempenho? |
| 02 | Impacto em iniciantes | Produzir mais é aprender mais? |
| 03 | Paradoxo do erro | O problema é o prompt ou o entendimento? |
| 04 | Preço neurológico | O que acontece quando terceirizamos o raciocínio? |
| 05 | Futuro profissional | O que sobra para o humano quando a IA escreve código? |

Vibe coding não é sinônimo de programar com IA

A diferença central é o quanto o humano revisa, testa e compreende o que foi gerado.

KARPATY · 2025

Vibe coding

- Aceita mudanças sem ler o diff
- Cola o erro de volta no chat
- Avalia sobretudo pelo comportamento visível

"forget that the code even exists"

WILLISON · 2025

Programação assistida por IA

- Revisa o diff e testa de verdade
- Consegue explicar o que vai commitar
- Usa a IA como ferramenta, não como substituto do entendimento

Regra prática: não commitar o que você não consegue explicar.

Fontes: Karpathy (2025); Willison (2025). O estudo-âncora analisa o "vibe coding puro": participantes não viam nem editavam o código.

O experimento: quem se sai melhor quando o código fica escondido?

Thorgeirsson, Weidmann & Su — CHI 2026 · 100 estudantes universitários em Zurique.

100

estudantes

3 × 15 min

tarefas de vibe coding

Claude

Sonnet 4

0 linhas

de código visíveis

SCS1

Conhecimento de CS
· 25 min

Redação

Escrita técnica · 20
min

ICAR16

Raciocínio geral · 12
min

O código era borrado: o participante precisava especificar, observar o resultado e orientar a IA sem "salvar" a solução editando manualmente.

Fundamento pesa mais do que “saber conversar com a IA”

Conhecimento de computação explicou aproximadamente o dobro da variação adicional de desempenho em relação à escrita.

Variação adicional explicada

Programação / CS



Escrita



≈ 2× mais peso para o fundamento

Uso frequente de LLM

$$r = -0,258$$

Quem relatava usar mais IA teve desempenho pior no experimento.

Familiaridade ≠ proficiência

Fonte: Thorgeirsson, Weidmann & Su (2026). CS + escrita explicaram 20,8% da variação; frequência de uso de LLM teve correlação negativa com desempenho.

IA na programação introdutória: o desempenho imediato

Kazemitabaar et al. (2023) · CHI '23

COM ACESSO À IA

↑ **Taxa de conclusão**

↓ **Tempo em erros sintáticos**

A ferramenta reduz bloqueios iniciais e acelera a execução da tarefa.

Efeito primário

Redução da barreira inicial de escrita de código

A IA ajuda a chegar mais rápido a uma solução — especialmente quando sintaxe e detalhes de implementação ainda travam o iniciante.

Fonte: Kazemitabaar et al. (2023), Studying the Effect of AI Code Generators on Supporting Novice Learners in Introductory Programming.

O teste de retenção: o que sobra quando a IA sai?

Produtividade durante a tarefa e aprendizagem consolidada não são a mesma coisa.

SEM ACESSO À IA

- Alunos com uso passivo apresentaram menor retenção lógica em testes sem a ferramenta.
- Houve queda no entendimento da estrutura interna do código gerado.
- A dependência pode mascarar lacunas de aprendizagem enquanto a ferramenta está disponível.

PRODUTIVIDADE



APRENDIZADO

Consolidar exige compreender, recuperar e reproduzir sem depender da resposta pronta.

Síntese baseada em Kazemitabaar et al. (2023): ganhos de execução com IA não garantem retenção quando o suporte é removido.

O iniciante falha por “falar errado” ou por não saber o que dizer?

Lucchetti et al. (2025) · NAACL — 1.749 prompts de 80 estudantes iniciantes.

ESTILO

Trocar palavras não resolve

Corrigir vocabulário técnico não trouxe ganhos confiáveis. O problema não é simplesmente “prompt engineering”.

Exceção: termos que induzem comportamento errado podem prejudicar o modelo.

SUBSTÂNCIA

A informação certa muda tudo

86%

sucesso com todas as pistas

40%

com 1 pista ausente

11%

só reescrevendo

A habilidade crítica é perceber qual informação está faltando — e isso exige entender o problema.

Fonte: Lucchetti et al. (2025), Substance Beats Style. Trajetórias com ciclos de tentativa tiveram 30% de sucesso vs. 72% sem ciclo.

Quando a IA reduz o esforço, ela pode reduzir também parte do aprendizado

O risco não é usar IA — é usá-la para evitar o processo de compreender, testar e corrigir.

1 · OFFLOADING

Terceirização do raciocínio

Menos esforço cognitivo pode significar menos construção ativa de conhecimento quando a ferramenta substitui a tentativa.

2 · FALSA CONFIANÇA

Funcionar não significa estar correto

Perry et al. encontraram mais código inseguro e, ao mesmo tempo, maior confiança de quem usou IA.

3 · AUTONOMIA

Aprender é passar pelo processo

Tentar → errar → investigar → corrigir → entender. Pular todas as etapas pode aumentar a produção sem desenvolver autonomia.

“Se a IA escreve o código, a principal habilidade do programador passa a ser saber quando aquele código está errado.”

Se a IA escreve o código, o rigor muda de lugar

O trabalho humano migra da digitação para especificação, supervisão, verificação e compreensão do sistema.

1

Decomposição

Dividir problemas em pacotes adequados para agentes.

2

Calibração de confiança

Saber quanto confiar e reconhecer respostas plausíveis, porém erradas.

3

Coerência arquitetural

Manter um modelo mental do sistema enquanto várias frentes mudam em paralelo.

4

Avaliação de qualidade

Julgar resultados sem depender da leitura minuciosa de cada linha.

O “middle loop” é a engenharia de supervisão: a IA executa mais, mas alguém ainda precisa decidir o que pedir, o que aceitar e o que rejeitar.

Fonte: Thoughtworks, The Future of Software Engineering Retreat Findings and Strategic Insights (2026).

A formação continua necessária — talvez mais do que antes

O risco não é a máquina escrever código. É a equipe perder a capacidade de entender o sistema que está crescendo.

JÚNIOR

IA pode acelerar a fase inicial

Mas as tarefas que ensinavam fundamentos precisam ser substituídas por outras formas de prática — não simplesmente eliminadas.

PLENO

Fundamento incompleto vira gargalo

O profissional que só traduz tarefa em código perde espaço; cresce a importância de arquitetura, verificação e supervisão.

EQUIPE

Dívida cognitiva

A distância entre a complexidade do software e o entendimento que as pessoas têm dele pode crescer mais rápido do que a dívida técnica tradicional.

IA assume a escrita do código — não a responsabilidade por ele.

Fonte: Thoughtworks (2026); relação com dívida cognitiva também discutida por Kosmyrna et al. (2025).

Então, a IA ajuda ou atrapalha quem está aprendendo?

As evidências apontam para uma resposta condicional: depende de como a ferramenta entra no processo de aprendizagem.

AJUDA QUANDO...

- reduz barreiras mecânicas sem substituir o raciocínio
- explica e oferece feedback durante a tentativa
- o aluno testa, questiona e consegue reproduzir a lógica
- serve como ferramenta para explorar hipóteses

ATRAPALHA QUANDO...

- entrega respostas prontas que o aluno apenas aceita
- elimina a necessidade de investigar erros
- cria confiança sem capacidade de validação
- transforma produtividade aparente em dependência

A questão não é “usar IA ou não”. É preservar o esforço cognitivo que constrói autonomia.

O bom programador com IA não é quem gera mais código.

É quem consegue transformar conhecimento em especificação e, depois, transformar a saída da IA em uma decisão confiável.

1 Fundamentos

entender lógica, estado, estruturas e limites

2 Especificação

decompor e dizer exatamente o que precisa acontecer

3 Validação

testar, detectar erro e calibrar confiança

4 Arquitetura

manter coerência e compreender o sistema

5 Autonomia

usar a IA sem terceirizar completamente o raciocínio

"A IA pode acelerar a execução. O aprendizado ainda depende de compreender."

Estudos e fontes principais

- 01 THORGEIRSSON, S.; WEIDMANN, T. B.; SU, Z. Computer Science Achievement and Writing Skills Predict Vibe Coding Proficiency. CHI 2026.
- 02 KAZEMITABAAR, M. et al. Studying the Effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. CHI 2023.
- 03 LUCCHETTI, F. et al. Substance Beats Style: Why Beginning Students Fail to Code with LLMs. NAACL 2025.
- 04 KOSMYNA, N. et al. Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing. 2025.
- 05 PERRY, N. et al. Do Users Write More Insecure Code with AI Assistants? ACM CCS 2023.
- 06 RISKÓ, E. F.; GILBERT, S. J. Cognitive offloading. Trends in Cognitive Sciences, 2016.
- 07 THOUGHTWORKS. The Future of Software Engineering Retreat Findings and Strategic Insights. 2026.
- 08 KARPATY, A. Publicação que cunhou "vibe coding". 2025 · WILLISON, S. Not all AI-assisted programming is vibe coding. 2025.